Nama: Muhammad Zainur Roziqin

Kelas: TI-2C

Absen: 17

JOBSHEET 9

Abstract Class dan Interface

PERCOBAAN 1: Abstract Class

Skenario: semua hewan punya karakteristik yang sama (umur bertambah satu setiap tahun), tetapi cara bergeraknya berbeda-beda. Karena itu *Hewan* dibuat sebagai abstract class, sedangkan cara bergerak dituliskan oleh masing-masing class turunannya.

Class Hewan (abstract class):

Hewan.java

```java
package abstractclass;

public abstract class Hewan {
    protected int umur;

    protected Hewan() {
        this.umur = 0;
    }

    public void bertambahUmur() {
        this.umur += 1;
    }

    public abstract void bergerak();
}
```

Hewan berisi atribut *umur*, constructor, dan method biasa *bertambahUmur()* yang langsung diwariskan, ditambah abstract method *bergerak()* yang tidak punya badan sehingga wajib di-override oleh setiap turunan.

Class Kucing:

Kucing.java

```java
package abstractclass;

public class Kucing extends Hewan {
    @Override
    public void bergerak() {
        System.out.println("Berjalan dengan KAKI, \"tap..tap..\"");
    }
}
```

Kucing meng-extend Hewan dan mengisi *bergerak()* dengan cara berjalan memakai kaki.

Class Ikan:

Ikan.java

```java
package abstractclass;

public class Ikan extends Hewan {
    @Override
    public void bergerak() {
        System.out.println("Berenang dengan SIRIP, \"wush..wush..\"");
    }
}
```

Ikan juga meng-extend Hewan, tetapi mengisi *bergerak()* dengan cara berenang memakai sirip.

Class Orang:

Orang.java

```java
package abstractclass;

public class Orang {
    private String nama;
    private Hewan hewanPeliharaan;

    public Orang(String nama) {
        this.nama = nama;
    }

    public void peliharaHewan(Hewan hewan) {
        this.hewanPeliharaan = hewan;
    }

    public void ajakPeliharaanJalanJalan() {
        System.out.println("Namaku " + this.nama);
        System.out.println("Hewan peliharaanku berjalan dengan cara: ");
        this.hewanPeliharaan.bergerak();
        System.out.println("----------------------------------------");
    }
}
```

Orang adalah pengguna Hewan. Atribut *hewanPeliharaan* bertipe Hewan (abstract class), sehingga dapat menampung objek turunan apa pun, baik Kucing maupun Ikan.

Class Program (main class):

Program.java

```java
package abstractclass;

public class Program {
    public static void main(String[] args) {
        Kucing kucingKampung = new Kucing();
        Ikan lumbaLumba = new Ikan();

        Orang ani = new Orang("Ani");
        Orang budi = new Orang("Budi");

        ani.peliharaHewan(kucingKampung);
        budi.peliharaHewan(lumbaLumba);

        ani.ajakPeliharaanJalanJalan();
        budi.ajakPeliharaanJalanJalan();
    }
}
```

Program membuat objek Kucing dan Ikan, dua objek Orang (Ani dan Budi), lalu memasangkan peliharaan dan mengajaknya berjalan-jalan.

Hasil:

Output - Program (run)

```
Namaku Ani
Hewan peliharaanku berjalan dengan cara:
Berjalan dengan KAKI, "tap..tap.."
---------------------------------------
Namaku Budi
Hewan peliharaanku berjalan dengan cara:
Berenang dengan SIRIP, "wush..wush.."
---------------------------------------
```

Jawaban Pertanyaan Percobaan 1

1. Jelaskan jalannya program di atas. Mengapa pemanggilan *this.hewanPeliharaan.bergerak()* di class Orang menghasilkan output yang berbeda untuk Ani dan Budi?

Program dimulai dari *main()*: dibuat objek Kucing, objek Ikan, serta objek Orang Ani dan Budi. *ani.peliharaHewan(kucingKampung)* menyimpan objek Kucing ke atribut *hewanPeliharaan* milik Ani, dan *budi.peliharaHewan(lumbaLumba)* menyimpan objek Ikan ke atribut milik Budi. Setelah itu *ajakPeliharaanJalanJalan()* dipanggil pada masing-masing orang.

Variabel *hewanPeliharaan* bertipe Hewan, tetapi objek yang sebenarnya ditunjuk berbeda: milik Ani adalah Kucing, milik Budi adalah Ikan. Saat program berjalan, Java memanggil method *bergerak()* sesuai objek aslinya (polimorfisme), sehingga Ani menghasilkan "Berjalan dengan KAKI" dan Budi menghasilkan "Berenang dengan SIRIP", walaupun baris kode pemanggilannya sama.

2. Tambahkan baris *Hewan hewan = new Hewan();* pada method main(). Tunjukkan hasil kompilasinya dan jelaskan.

Program.java (ditambahkan di mai

```java
Hewan hewan = new Hewan();
```

Output - javac

```
\abstractclass\Program.java:5: error: Hewan is abstract; cannot be instantiated
        Hewan hewan = new Hewan();
                      ^
1 error
```

Kompilasi gagal karena Hewan adalah abstract class. Abstract class belum lengkap (ada method *bergerak()* yang belum punya badan), sehingga tidak boleh dibuat objeknya dengan *new*. Objek harus dibuat dari class turunan yang sudah mengimplementasikan seluruh abstract method, misalnya *new Kucing()*.

3. Bolehkah sebuah class yang meng-extend abstract class tidak mengimplementasikan abstract method induknya? Buktikan dengan menghapus method *bergerak()* dari class Ikan.

Ikan.java (method bergerak() dihapus)

```java
package abstractclass;

public class Ikan extends Hewan {
}
```

Output - javac

```
\abstractclass\Ikan.java:3: error: Ikan is not abstract and does not override abstract method bergerak() in Hewan
public class Ikan extends Hewan {
       ^
1 error
```

Tidak boleh. Class turunan yang bukan abstract wajib meng-override semua abstract method induknya. Compiler menolak dengan pesan "Ikan is not abstract and does not override abstract method bergerak() in Hewan". Satu-satunya cara agar boleh tidak mengimplementasikan adalah dengan menjadikan Ikan sendiri sebagai abstract class.

4. Tambahkan abstract method *public abstract void bernapas();* di dalam class Ikan tanpa mengubah deklarasi class Ikan.

Ikan.java (ditambahkan)

```java
public class Ikan extends Hewan {
    public abstract void bernapas();
    ...
```

Output - javac

```
\abstractclass\Ikan.java:3: error: Ikan is not abstract and does not override abstract method bernapas() in Ikan
public class Ikan extends Hewan {
       ^
1 error
```

Kompilasi gagal. Aturan yang dilanggar: class yang memiliki minimal satu abstract method wajib dideklarasikan sebagai abstract class. Ikan masih dideklarasikan sebagai class biasa, sehingga dianggap kurang lengkap. Perbaikannya adalah mengubah deklarasi menjadi *public abstract class Ikan*, tetapi konsekuensinya Ikan tidak bisa lagi dibuat objeknya dengan *new*.

PERCOBAAN 2: Interface

Skenario: pada wisuda, Rektor memberi sertifikat Cumlaude kepada mahasiswa yang memenuhi syarat. Syarat tersebut dituangkan dalam interface *ICumlaude* yang bisa dipenuhi oleh Sarjana maupun PascaSarjana.

Interface ICumlaude:

ICumlaude.java

```java
package interfacelatihan;

public interface ICumlaude {
    public abstract void lulus();
    public abstract void meraihIPKTinggi();
}
```

Interface berisi dua abstract method sebagai kontrak: *lulus()* dan *meraihIPKTinggi()*.

Class Mahasiswa:

Mahasiswa.java

```java
package interfacelatihan;

public class Mahasiswa {
    protected String nama;

    public Mahasiswa(String nama) {
        this.nama = nama;
    }

    public void kuliahDiKampus() {
        System.out.println("Aku mahasiswa, namaku " + this.nama);
        System.out.println("Aku berkuliah di kampus.");
    }
}
```

Class induk biasa yang memiliki atribut *nama* dan method *kuliahDiKampus()*.

Class Sarjana:

Sarjana.java

```java
package interfacelatihan;

public class Sarjana extends Mahasiswa implements ICumlaude {
    public Sarjana(String nama) {
        super(nama);
    }

    @Override
    public void lulus() {
        System.out.println("Aku sudah menyelesaikan SKRIPSI");
    }

    @Override
    public void meraihIPKTinggi() {
        System.out.println("IPK-ku lebih dari 3,51");
    }
}
```

Sarjana meng-extend Mahasiswa (hubungan "adalah") dan meng-implements ICumlaude (hubungan "dapat"). Method interface ditulis dengan *public* dan *@Override*.

Class PascaSarjana:

PascaSarjana.java

```java
package interfacelatihan;

public class PascaSarjana extends Mahasiswa implements ICumlaude {
    public PascaSarjana(String nama) {
        super(nama);
    }

    @Override
    public void lulus() {
        System.out.println("Aku sudah menyelesaikan TESIS");
    }

    @Override
    public void meraihIPKTinggi() {
        System.out.println("IPK-ku lebih dari 3,71");
    }
}
```

Sama seperti Sarjana, tetapi syaratnya berbeda: menyelesaikan tesis dan IPK lebih dari 3,71.

Class Rektor:

Rektor.java

```java
package interfacelatihan;

public class Rektor {
    public void beriSertifikatCumlaude(ICumlaude mahasiswaCumlaude) {
        System.out.println("Saya REKTOR, memberikan sertifikat cumlaude.");
        System.out.println("Selamat! Silakan perkenalkan diri Anda..");

        mahasiswaCumlaude.lulus();
        mahasiswaCumlaude.meraihIPKTinggi();

        System.out.println("----------------------------------------");
    }
}
```

Parameter *beriSertifikatCumlaude()* bertipe interface *ICumlaude*, sehingga Rektor dapat menerima objek apa pun yang meng-implements ICumlaude tanpa peduli class induknya.

Class Program (setelah diperbaiki):

Program.java

```java
package interfacelatihan;

public class Program {
    public static void main(String[] args) {
        Rektor pakRektor = new Rektor();

        Mahasiswa mhsBiasa = new Mahasiswa("Charlie");
        Sarjana sarjanaCumlaude = new Sarjana("Dini");
        PascaSarjana masterCumlaude = new PascaSarjana("Elok");

        // Perbaikan: mhsBiasa tidak meng-implements ICumlaude, sehingga tidak boleh diberi sertifikat
        // pakRektor.beriSertifikatCumlaude(mhsBiasa);
        pakRektor.beriSertifikatCumlaude(sarjanaCumlaude);
        pakRektor.beriSertifikatCumlaude(masterCumlaude);
    }
}
```

Pada kode awal terdapat pemanggilan *beriSertifikatCumlaude(mhsBiasa)* yang menyebabkan error (dijelaskan pada pertanyaan 1). Baris tersebut dijadikan komentar agar program dapat berjalan.

Hasil:

Output - Program (run)

```
Saya REKTOR, memberikan sertifikat cumlaude.
Selamat! Silakan perkenalkan diri Anda..
Aku sudah menyelesaikan SKRIPSI
IPK-ku lebih dari 3,51
---------------------------------------
Saya REKTOR, memberikan sertifikat cumlaude.
Selamat! Silakan perkenalkan diri Anda..
Aku sudah menyelesaikan TESIS
IPK-ku lebih dari 3,71
---------------------------------------
```

Jawaban Pertanyaan Percobaan 2

1. Mengapa pada langkah 7 terjadi error? Tuliskan pesan error-nya dan jelaskan bagaimana Anda memperbaikinya.

Program.java (kode awal langkah 6)

```java
pakRektor.beriSertifikatCumlaude(mhsBiasa);
```

Output - javac

```
\interfacelatihan\Program.java:12: error: incompatible types: Mahasiswa cannot be converted to ICumlaude
        pakRektor.beriSertifikatCumlaude(mhsBiasa);
                                         ^
Note: Some messages have been simplified; recompile with -Xdiags:verbose to get full output
1 error
```

Error terjadi karena *mhsBiasa* bertipe Mahasiswa, sedangkan parameter method menuntut tipe ICumlaude. Class Mahasiswa tidak meng-implements ICumlaude, sehingga tidak memenuhi kontrak dan tidak bisa dikonversi. Perbaikannya: menghapus (menjadikan komentar) baris pemanggilan untuk *mhsBiasa*, karena mahasiswa biasa memang tidak berhak menerima sertifikat

Cumlaude. Alternatif lain, Mahasiswa dibuat meng-implements ICumlaude, tetapi itu kurang tepat karena tidak semua mahasiswa cumlaude.

2. Dapatkah method *kuliahDiKampus()* dipanggil dari objek *sarjanaCumlaude* di class Program? Mengapa demikian?

Program.java (ditambahkan di main)

```
1 sarjanaCumlaude.kuliahDiKampus();
```

Output - Program (run)

```
Aku mahasiswa, namaku Dini
Aku berkuliah di kampus.
Saya REKTOR, memberikan sertifikat cumlaude.
Selamat! Silakan perkenalkan diri Anda..
Aku sudah menyelesaikan SKRIPSI
IPK-ku lebih dari 3,51
---------------------------------------
Saya REKTOR, memberikan sertifikat cumlaude.
Selamat! Silakan perkenalkan diri Anda..
Aku sudah menyelesaikan TESIS
IPK-ku lebih dari 3,71
---------------------------------------
```

Bisa. Variabel *sarjanaCumlaude* bertipe Sarjana, dan Sarjana adalah turunan Mahasiswa sehingga mewarisi method *kuliahDiKampus()* yang berstatus public.

3. Dapatkah method *kuliahDiKampus()* dipanggil dari parameter *mahasiswaCumlaude* di method *beriSertifikatCumlaude()* pada class Rektor? Mengapa demikian?

Rektor.java (ditambahkan)

```
1 mahasiswaCumlaude.kuliahDiKampus();
```

Output - javac

```
\interfacelatihan\Rektor.java:8: error: cannot find symbol
        mahasiswaCumlaude.kuliahDiKampus();
                         ^
  symbol:   method kuliahDiKampus()
  location: variable mahasiswaCumlaude of type ICumlaude
1 error
```

Tidak bisa. Parameter *mahasiswaCumlaude* bertipe ICumlaude, sehingga yang dikenali hanya method yang ada di interface tersebut, yaitu *lulus()* dan *meraihIPKTinggi()*. Method *kuliahDiKampus()* adalah milik class Mahasiswa, bukan bagian dari kontrak ICumlaude.

4. Hapus kata kunci *public* pada method *lulus()* di class Sarjana. Tunjukkan hasil kompilasinya dan jelaskan.

Sarjana.java (public dihapus)

```
1     @Override
2     void lulus() {
3         System.out.println("Aku sudah menyelesaikan SKRIPSI");
4     }
```

```
Output - javac

\interfacelatihan\Sarjana.java:9: error: lulus() in Sarjana cannot implement lulus() in ICumlaude
    void lulus() {
         ^
  attempting to assign weaker access privileges; was public
1 error
```

Kompilasi gagal dengan pesan "attempting to assign weaker access privileges; was public". Method di interface otomatis public, sedangkan method tanpa modifier hanya berstatus package-private (akses lebih lemah). Method implementasi tidak boleh mempersempit akses, sehingga wajib ditulis *public*. Kode sudah dikembalikan seperti semula.

PERCOBAAN 3: Multiple Interface

Skenario: Rektor juga memberi sertifikat Mawapres (Mahasiswa Berprestasi) dengan kriteria menjuarai kompetisi dan membuat publikasi ilmiah. Kriteria ini dituangkan dalam interface baru *IBerprestasi*, sehingga satu class dapat meng-implements lebih dari satu interface.

Interface IBerprestasi:

IBerprestasi.java

```java
package interfacelatihan;

public interface IBerprestasi {
    public abstract void menjuaraiKompetisi();
    public abstract void membuatPublikasiIlmiah();
}
```

Interface baru berisi *menjuaraiKompetisi()* dan *membuatPublikasiIlmiah()*.

Class PascaSarjana (multiple interface):

PascaSarjana.java

```java
package interfacelatihan;

public class PascaSarjana extends Mahasiswa implements ICumlaude, IBerprestasi {
    public PascaSarjana(String nama) {
        super(nama);
    }

    @Override
    public void lulus() {
        System.out.println("Aku sudah menyelesaikan TESIS");
    }

    @Override
    public void meraihIPKTinggi() {
        System.out.println("IPK-ku lebih dari 3,71");
    }

    @Override
    public void menjuaraiKompetisi() {
        System.out.println("Saya telah menjuarai kompetisi INTERNASIONAL");
    }

    @Override
    public void membuatPublikasiIlmiah() {
        System.out.println("Saya menerbitkan artikel di jurnal INTERNASIONAL");
    }
}
```

PascaSarjana meng-implements dua interface sekaligus (dipisah koma): ICumlaude dan IBerprestasi, sehingga wajib mengisi keempat method dari kedua interface.

Class Rektor:

Rektor.java

```java
package interfacelatihan;

public class Rektor {
    public void beriSertifikatCumlaude(ICumlaude mahasiswaCumlaude) {
        System.out.println("Saya REKTOR, memberikan sertifikat cumlaude.");
        System.out.println("Selamat! Silakan perkenalkan diri Anda..");

        mahasiswaCumlaude.lulus();
        mahasiswaCumlaude.meraihIPKTinggi();

        System.out.println("----------------------------------------");
    }

    public void beriSertifikatMawapres(IBerprestasi mahasiswaBerprestasi) {
        System.out.println("Saya REKTOR, memberikan sertifikat MAWAPRES.");
        System.out.println("Selamat! Bagaimana Anda bisa berprestasi?");

        mahasiswaBerprestasi.menjuaraiKompetisi();
        mahasiswaBerprestasi.membuatPublikasiIlmiah();

        System.out.println("----------------------------------------");
    }
}
```

Ditambahkan method *beriSertifikatMawapres()* yang parameternya bertipe interface IBerprestasi.

Class Program:

Program.java

```java
package interfacelatihan;

public class Program {
    public static void main(String[] args) {
        Rektor pakRektor = new Rektor();

        Sarjana sarjanaCumlaude = new Sarjana("Dini");
        PascaSarjana masterCumlaude = new PascaSarjana("Elok");

        // pakRektor.beriSertifikatCumlaude(sarjanaCumlaude);
        // pakRektor.beriSertifikatCumlaude(masterCumlaude);

        pakRektor.beriSertifikatMawapres(sarjanaCumlaude);
        pakRektor.beriSertifikatMawapres(masterCumlaude);
    }
}
```

Pemanggilan *beriSertifikatCumlaude()* dijadikan komentar, lalu ditambahkan pemanggilan *beriSertifikatMawapres()* untuk sarjanaCumlaude dan masterCumlaude.

Error pada langkah 4 dan perbaikannya

Jika Sarjana belum meng-implements IBerprestasi, kompilasi gagal seperti di bawah ini. Penyebabnya, *beriSertifikatMawapres()* hanya menerima objek bertipe IBerprestasi, sedangkan Sarjana belum memenuhi kontrak itu. Perbaikannya adalah membuat Sarjana ikut meng-implements IBerprestasi (lihat Pertanyaan 3). Jika hanya ingin menghasilkan output persis seperti contoh pada langkah 5, cukup memanggil *beriSertifikatMawapres()* untuk *masterCumlaude* saja.

```
Output - javac

\interfacelatihan\Program.java:13: error: incompatible types: Sarjana cannot be converted to IBerprestasi
        pakRektor.beriSertifikatMawapres(sarjanaCumlaude);
                                         ^
Note: Some messages have been simplified; recompile with -Xdiags:verbose to get full output
1 error
```

Class Sarjana (setelah diperbaiki):

Sarjana.java

```java
package interfacelatihan;

// Pertanyaan 3: Sarjana juga meng-implements IBerprestasi (kompetisi dan jurnal tingkat NASIONAL)
public class Sarjana extends Mahasiswa implements ICumlaude, IBerprestasi {
    public Sarjana(String nama) {
        super(nama);
    }

    @Override
    public void lulus() {
        System.out.println("Aku sudah menyelesaikan SKRIPSI");
    }

    @Override
    public void meraihIPKTinggi() {
        System.out.println("IPK-ku lebih dari 3,51");
    }

    @Override
    public void menjuaraiKompetisi() {
        System.out.println("Saya telah menjuarai kompetisi NASIONAL");
    }

    @Override
    public void membuatPublikasiIlmiah() {
        System.out.println("Saya menerbitkan artikel di jurnal NASIONAL");
    }
}
```

Sarjana meng-implements ICumlaude dan IBerprestasi, dengan isi method bertema NASIONAL sesuai Pertanyaan 3.

Hasil:

```
Output - Program (run)

Saya REKTOR, memberikan sertifikat MAWAPRES.
Selamat! Bagaimana Anda bisa berprestasi?
Saya telah menjuarai kompetisi NASIONAL
Saya menerbitkan artikel di jurnal NASIONAL
---------------------------------------
Saya REKTOR, memberikan sertifikat MAWAPRES.
Selamat! Bagaimana Anda bisa berprestasi?
Saya telah menjuarai kompetisi INTERNASIONAL
Saya menerbitkan artikel di jurnal INTERNASIONAL
---------------------------------------
```

Jawaban Pertanyaan Percobaan 3

1. Pada interface IBerprestasi, ganti modifier *public* pada *menjuaraiKompetisi()* menjadi *protected* dan pada *membuatPublikasiIlmiah()* menjadi *private*. Apa yang terjadi?

IBerprestasi.java (diubah)

```
1     protected abstract void menjuaraiKompetisi();
2     private abstract void membuatPublikasiIlmiah();
```

Output - javac

```
\interfacelatihan\IBerprestasi.java:4: error: modifier protected not allowed here
    protected abstract void menjuaraiKompetisi();
                            ^
\interfacelatihan\IBerprestasi.java:5: error: illegal combination of modifiers: abstract and private
    private abstract void membuatPublikasiIlmiah();
                          ^
\interfacelatihan\PascaSarjana.java:23: error: method does not override or implement a method from a supertype
    @Override
    ^
\interfacelatihan\Rektor.java:19: error: membuatPublikasiIlmiah() has private access in IBerprestasi
        mahasiswaBerprestasi.membuatPublikasiIlmiah();
                            ^
\interfacelatihan\Sarjana.java:24: error: method does not override or implement a method from a supertype
    @Override
    ^
5 errors
```

Kompilasi gagal. Method interface tidak boleh *protected* ("modifier protected not allowed here") dan tidak boleh *private abstract* ("illegal combination of modifiers: abstract and private"). Alasannya, interface adalah kontrak yang harus bisa dipanggil dari luar dan di-override oleh class lain, jadi method-nya harus public. Method private tidak bisa di-override, sehingga bertentangan dengan sifat abstract. Error lain dalam daftar (@Override dan akses private di Rektor) hanyalah akibat turunan dari dua kesalahan itu. Kode sudah dikembalikan seperti semula.

2. Tambahkan baris *IBerprestasi prestasi = new IBerprestasi();* pada main(). Mengapa baris tersebut error?

Program.java (ditambahkan di main)

```
1 IBerprestasi prestasi = new IBerprestasi();
```

Output - javac

```
\interfacelatihan\Program.java:5: error: IBerprestasi is abstract; cannot be instantiated
        IBerprestasi prestasi = new IBerprestasi();
                                    ^
1 error
```

Interface tidak dapat diinstansiasi karena hanya berisi deklarasi method tanpa implementasi dan tidak memiliki constructor. Objek harus dibuat dari class yang meng-implements-nya, misalnya *IBerprestasi prestasi = new PascaSarjana("Elok");*.

3. Apabila Sarjana Berprestasi harus menjuarai kompetisi NASIONAL dan menerbitkan artikel di jurnal NASIONAL, modifikasilah class-class yang terkait.

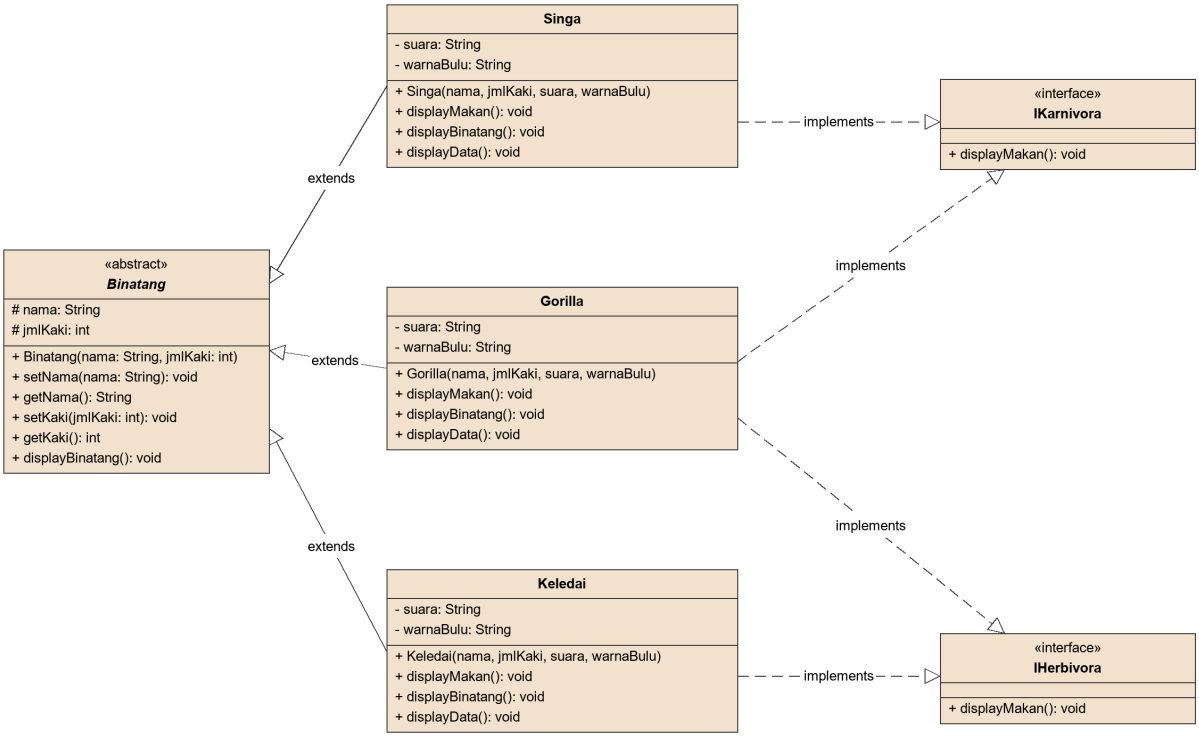
Class yang dimodifikasi hanya **Sarjana**: ditambahkan *IBerprestasi* setelah *implements* dan diisi kedua method-nya dengan teks NASIONAL (kode Sarjana ada di bagian sebelumnya). Class

Rektor dan Program tidak perlu diubah karena Rektor bekerja lewat tipe interface. Hasil eksekusinya sama dengan output Percobaan 3 di atas, yaitu Dini bertema NASIONAL dan Elok bertema INTERNASIONAL.

TUGAS: Binatang, IKarnivora, dan IHerbivora

Singa hanya makan daging (Karnivora), Keledai hanya makan tumbuhan (Herbivora), dan Gorilla makan keduanya (Karnivora + Herbivora). *Binatang* dibuat sebagai abstract class karena hubungannya "adalah" (Singa adalah Binatang), sedangkan *IKarnivora* dan *IHerbivora* berupa interface karena hubungannya "dapat". Gorilla meng-implements kedua interface sekaligus.

UML Class Diagram



Gambar 1. Class diagram sistem Binatang.

Class Binatang (abstract class):

Binatang.java

```java
package tugas;

public abstract class Binatang {
    protected String nama;
    protected int jmlKaki;

    public Binatang(String nama, int jmlKaki) {
        this.nama = nama;
        this.jmlKaki = jmlKaki;
    }

    public void setNama(String nama) {
        this.nama = nama;
    }

    public String getNama() {
        return nama;
    }

    public void setKaki(int jmlKaki) {
        this.jmlKaki = jmlKaki;
    }

    public int getKaki() {
        return jmlKaki;
    }

    public void displayBinatang() {
        System.out.println("Nama: " + nama);
        System.out.println("Jumlah Kaki: " + jmlKaki);
    }
}
```

Berisi atribut *nama* dan *jmlKaki* (protected), constructor, getter/setter, dan method *displayBinatang()* yang mencetak nama dan jumlah kaki untuk semua binatang.

Interface IKarnivora:

IKarnivora.java

```java
package tugas;

public interface IKarnivora {
    public abstract void displayMakan();
}
```

Kontrak untuk pemakan daging: *displayMakan()*.

Interface IHerbivora:

IHerbivora.java

```java
package tugas;

public interface IHerbivora {
    public abstract void displayMakan();
}
```

Kontrak untuk pemakan tumbuhan, dengan method *displayMakan()* bersignature sama dengan IKarnivora.

Class Singa:

Singa.java

```java
package tugas;

public class Singa extends Binatang implements IKarnivora {
    private String suara;
    private String warnaBulu;

    public Singa(String nama, int jmlKaki, String suara, String warnaBulu) {
        super(nama, jmlKaki);
        this.suara = suara;
        this.warnaBulu = warnaBulu;
    }

    @Override
    public void displayMakan() {
        System.out.println("Jenis: Karnivora");
        System.out.println("Makanan: Daging");
    }

    @Override
    public void displayBinatang() {
        super.displayBinatang();
        System.out.println("Suara: " + suara);
        System.out.println("Warna Bulu: " + warnaBulu);
    }

    public void displayData() {
        displayMakan();
        displayBinatang();
    }
}
```

Meng-extend Binatang dan meng-implements IKarnivora. *displayBinatang()* di-override untuk menambah suara dan warna bulu, sedangkan *displayData()* menampilkan jenis makanan lalu data binatang.

Class Keledai:

Keledai.java

```java
package tugas;

public class Keledai extends Binatang implements IHerbivora {
    private String suara;
    private String warnaBulu;

    public Keledai(String nama, int jmlKaki, String suara, String warnaBulu) {
        super(nama, jmlKaki);
        this.suara = suara;
        this.warnaBulu = warnaBulu;
    }

    @Override
    public void displayMakan() {
        System.out.println("Jenis: Herbivora");
        System.out.println("Makanan: Tumbuhan");
    }

    @Override
    public void displayBinatang() {
        super.displayBinatang();
        System.out.println("Suara: " + suara);
        System.out.println("Warna Bulu: " + warnaBulu);
    }

    public void displayData() {
        displayMakan();
        displayBinatang();
    }
}
```

Meng-extend Binatang dan meng-implements IHerbivora, dengan struktur yang sama seperti Singa.

Class Gorilla:

Gorilla.java

```java
package tugas;

public class Gorilla extends Binatang implements IKarnivora, IHerbivora {
    private String suara;
    private String warnaBulu;

    public Gorilla(String nama, int jmlKaki, String suara, String warnaBulu) {
        super(nama, jmlKaki);
        this.suara = suara;
        this.warnaBulu = warnaBulu;
    }

    // Satu method ini memenuhi displayMakan() milik IKarnivora sekaligus IHerbivora
    @Override
    public void displayMakan() {
        System.out.println("Jenis: Karnivora + Herbivora");
        System.out.println("Makanan: Daging + Tumbuhan");
    }

    @Override
    public void displayBinatang() {
        super.displayBinatang();
        System.out.println("Suara: " + suara);
        System.out.println("Warna Bulu: " + warnaBulu);
    }

    public void displayData() {
        displayMakan();
        displayBinatang();
    }
}
```

Meng-implements IKarnivora dan IHerbivora sekaligus. Karena kedua interface memiliki method *displayMakan()* dengan signature yang sama, cukup ditulis satu method yang memenuhi keduanya.

Class Program (main class):

Program.java

```java
package tugas;

public class Program {
    public static void main(String[] args) {
        Keledai keledai = new Keledai("Keledai", 4, "Hehehehe", "Abu-Abu");
        Gorilla gorilla = new Gorilla("Gulali", 4, "Haaum Hauum", "Hitam Manis");
        Singa singa = new Singa("CingaCing", 4, "Roaar Roaaar", "Coklat");

        keledai.displayData();
        System.out.println();
        gorilla.displayData();
        System.out.println();
        singa.displayData();
    }
}
```

Membuat objek Keledai, Gorilla, dan Singa dengan data sesuai soal, lalu memanggil *displayData()* masing-masing.

Hasil:

```
Output - Program (run)

Jenis: Herbivora
Makanan: Tumbuhan
Nama: Keledai
Jumlah Kaki: 4
Suara: Hehehehe
Warna Bulu: Abu-Abu

Jenis: Karnivora + Herbivora
Makanan: Daging + Tumbuhan
Nama: Gulali
Jumlah Kaki: 4
Suara: Haaum Hauum
Warna Bulu: Hitam Manis

Jenis: Karnivora
Makanan: Daging
Nama: CingaCing
Jumlah Kaki: 4
Suara: Roaar Roaaar
Warna Bulu: Coklat
```